

R - Control IP 4 – Webhook

Was ist das?

Der Webhook meldet Ereignisse des Geräts **sofort an eine Adresse Ihrer Wahl**. Immer wenn ein Ausgang schaltet oder jemand einen Taster drückt, schickt das Gerät eine kleine Nachricht per HTTP an Ihren Server.

Das ist der einfachste Weg, das Gerät an etwas anzubinden, das kein MQTT und kein Modbus spricht — zum Beispiel Node-RED, ioBroker, n8n, Zapier, eine Hausautomation oder ein selbstgeschriebenes Skript.

Der Unterschied zu den anderen Schnittstellen:

	Webhook	MQTT	Modbus/HTTP-API
Richtung	Gerät → Ihr Server	in beide Richtungen	Ihr Server fragt das Gerät
Sie brauchen	eine erreichbare URL	einen MQTT-Broker	nichts weiter
Gut für	„Sag mir Bescheid, wenn ...“	vollständige Integration	Zustand abfragen, schalten

Der Webhook **meldet nur**. Schalten können Sie darüber nicht — dafür nutzen Sie die REST-API, MQTT oder Modbus.

In 5 Minuten zum ersten Ergebnis

Schritt 1: Eine Adresse zum Testen

Wenn Sie noch keinen eigenen Empfänger haben, nutzen Sie zum Ausprobieren einen kostenlosen Test-Dienst wie webhook.site. Die Seite zeigt Ihnen sofort eine eigene URL, etwa:

```
https://webhook.site/8f14e45f-ceeda-467a-9f3b-1a2b3c4d5e6f
```

Schritt 2: Im Gerät eintragen

Öffnen Sie die Weboberfläche des Geräts:

Einstellungen → Schnittstellen → Webhook

1. Haken bei **Webhook aktivieren** setzen
2. **Ziel-URL** eintragen (die Adresse aus Schritt 1)
3. **Speichern** drücken
4. **Testnachricht senden** drücken

Im Browser-Tab von webhook.site erscheint jetzt die Testnachricht.

Schritt 3: Ein echtes Ereignis auslösen

Schalten Sie einen Ausgang — in der Weboberfläche unter **Schaltausgänge** oder direkt am Gerät. Sekunden später steht die Meldung bei Ihrem Empfänger:

```
{
  "device": "R-Control-IP4",
  "mac": "70:B3:D5:12:34:56",
  "event": "output",
  "action": "on",
  "index": 2,
  "name": "Hoflicht",
  "state": true,
  "timestamp_ms": 1789720000000,
  "uptime_s": 8641
}
```

Fertig. Alles Weitere in diesem Dokument ist Feinschliff.

Die Einstellungen im Einzelnen

Einstellungen → **Schnittstellen** → **Webhook**

Feld	Bedeutung
Webhook aktivieren	Schaltet die Funktion ein und aus
Ziel-URL	Wohin die Nachrichten gehen. Muss mit http:// oder https:// beginnen
Geheimnis (optional)	Wird als Kopfzeile X-Webhook-Secret mitgeschickt. Damit erkennt Ihr Server, dass die Nachricht wirklich von diesem Gerät kommt
Zeitlimit	Wie lange das Gerät auf eine Antwort wartet (1–30 Sekunden, Standard 5)
Schaltausgänge	Meldungen bei Ein-/Ausschalten der Ausgänge
Schalteingänge	Meldungen bei Tastendruck
Gerätestart	Eine Meldung, wenn das Gerät hochfährt

Bei **https://** prüft das Gerät das Zertifikat der Gegenstelle gegen die eingebauten Stammzertifikate. Ein selbstsigniertes Zertifikat auf Ihrem Server funktioniert daher **nicht** — nutzen Sie in dem Fall **http://** im lokalen Netz oder ein Zertifikat von einer bekannten Zertifizierungsstelle.

Wie eine Nachricht aussieht

Das Gerät sendet einen **HTTP-POST** mit **Content-Type: application/json**.

Ausgang geschaltet

```
{
  "device": "R-Control-IP4",
  "mac": "70:B3:D5:12:34:56",
  "event": "output",
  "action": "on",
  "index": 2,
  "name": "Hoflicht",
  "state": true,
  "timestamp_ms": 1789720000000,
  "uptime_s": 8641
}
```

Taster gedrückt

```
{
  "device": "R-Control-IP4",
  "mac": "70:B3:D5:12:34:56",
  "event": "input",
  "action": "click",
  "index": 1,
  "name": "Klingel",
  "timestamp_ms": 1789720012345,
  "uptime_s": 8653
}
```

Gerät gestartet

```
{
  "device": "R-Control-IP4",
  "mac": "70:B3:D5:12:34:56",
  "event": "system",
  "action": "boot",
  "timestamp_ms": 1789719991000,
  "uptime_s": 3
}
```

Die Felder

Feld	Immer dabei?	Bedeutung
device	ja	Hostname des Geräts (Einstellungen → Netzwerk)
mac	ja	MAC-Adresse — eindeutig, auch wenn mehrere Geräte denselben Namen haben
event	ja	output, input, system oder test

Feld	Immer dabei?	Bedeutung
<code>action</code>	ja	Was passiert ist, siehe Tabelle unten
<code>index</code>	bei <code>output/input</code>	Nummer des Ausgangs bzw. Eingangs (1–4)
<code>name</code>	bei <code>output/input</code>	Der in den Einstellungen vergebene Name
<code>state</code>	nur bei <code>output</code>	<code>true</code> = eingeschaltet, <code>false</code> = ausgeschaltet
<code>timestamp_ms</code>	wenn die Uhr gestellt ist	Zeitpunkt in Millisekunden seit 1970
<code>uptime_s</code>	ja	Wie lange das Gerät schon läuft, in Sekunden

Mögliche Werte für `action`:

event	action	Wann
<code>output</code>	<code>on / off</code>	Ausgang wurde ein- bzw. ausgeschaltet — egal ob per Taster, Zeitschaltuhr, App oder Hand
<code>input</code>	<code>pressed</code>	Taster wurde gedrückt
<code>input</code>	<code>released</code>	Taster wurde losgelassen
<code>input</code>	<code>long_pressed</code>	Taster wurde lange gedrückt
<code>input</code>	<code>click</code>	Kurzer Klick (drücken + loslassen)
<code>input</code>	<code>double_click</code>	Doppelklick
<code>system</code>	<code>boot</code>	Gerät ist gestartet
<code>test</code>	<code>test</code>	Sie haben „Testnachricht senden“ gedrückt

Hinweis zu `timestamp_ms`: Direkt nach dem Einschalten kennt das Gerät die Uhrzeit noch nicht. Bis die Zeit per NTP gestellt ist, fehlt das Feld. Verlassen Sie sich im Zweifel auf den Empfangszeitpunkt bei Ihrem Server.

Beispiele für Empfänger

Node-RED

1. **http in**-Knoten einfügen, Methode `POST`, URL z. B. `/rcontrol`
2. Dahinter einen **http response**-Knoten mit Statuscode `200`
3. Ziel-URL im Gerät: `http://<node-red-ip>:1880/rcontrol`

Vom **http in**-Knoten kommt die Nachricht in `msg.payload`. Ein Function-Knoten, der nur auf das Einschalten von Ausgang 2 reagiert:

```
if (msg.payload.event === "output" && msg.payload.index === 2 &&
msg.payload.state) {
```

```
    return msg;    // weiterleiten
}
return null;      // alles andere verwerfen
```

Home Assistant

In der `configuration.yaml`:

```
automation:
- alias: "Klingel gedrückt"
  trigger:
    platform: webhook
    webhook_id: rcontrol_ereignis
    allowed_methods: [POST]
    local_only: true
    condition: "{{ trigger.json.event == 'input' and trigger.json.action ==
'click' }}"
  action:
    - service: notify.mobile_app
      data:
        message: "Es hat geklingelt ({{ trigger.json.name }})"
```

Ziel-URL im Gerät: `http://<homeassistant-ip>:8123/api/webhook/rcontrol_ereignis`

Für reine Home-Assistant-Nutzer ist die **MQTT-Schnittstelle** meist die bessere Wahl: Sie legt die Geräte automatisch an und kann auch schalten. Der Webhook eignet sich, wenn Sie ohne Broker auskommen wollen.

Ein einfaches PHP-Skript

```
<?php
// empfaenger.php
$daten = json_decode(file_get_contents('php://input'), true);

// Geheimnis prüfen (optional, aber empfohlen)
$geheimnis = $_SERVER['HTTP_X_WEBHOOK_SECRET'] ?? '';
if ($geheimnis !== 'mein-geheimnis') {
    http_response_code(403);
    exit;
}

$zeile = sprintf("%s %s %s %s\n",
    date('Y-m-d H:i:s'),
    $daten['event'] ?? '?',
    $daten['name'] ?? '',
    $daten['action'] ?? '');
file_put_contents('ereignisse.log', $zeile, FILE_APPEND);
```

```
http_response_code(200);
```

Zum Ausprobieren auf dem eigenen Rechner

Ein Einzeiler, der eingehende Nachrichten anzeigt (Python muss installiert sein):

```
python -m http.server 8080
```

Das zeigt zwar nur die Anfragezeile, reicht aber, um zu sehen, **dass** etwas ankommt. Für den vollen Inhalt eignet sich webhook.site besser.

Was Ihr Server antworten sollte

Antworten Sie mit einem Statuscode **200–299**. Alles andere gilt als Fehler und erscheint in der Weboberfläche unter **Übersicht** → **Gerätestatus** als Hinweis.

Antworten Sie **schnell** — das Gerät wartet auf Ihre Antwort, standardmäßig bis zu 5 Sekunden. Lange Verarbeitung gehört hinter die Antwort, nicht davor.

Ein Antwort-Inhalt wird nicht ausgewertet; eine leere Antwort genügt.

Grenzen — bitte vorher lesen

Keine Wiederholung bei Fehlern. Ist Ihr Server gerade nicht erreichbar, geht diese eine Meldung verloren. Das Gerät versucht es nicht erneut.

Begrenzte Warteschlange. Bis zu 8 Meldungen warten auf Zustellung. Bei sehr schnellen Ereignisfolgen und einem langsamen Empfänger können Meldungen verworfen werden. Wie oft das passiert ist, steht unter **Übersicht** → **Gerätestatus** → **Details**.

Wenn Sie garantierte Zustellung brauchen, nutzen Sie MQTT. Ein Broker puffert Nachrichten und stellt sie nach einer Unterbrechung nach.

Der Webhook ist für den Normalfall gedacht: „Sag meinem System Bescheid, wenn etwas passiert.“ Für Abrechnungen, Sicherheitsprotokolle oder Ähnliches ist er nicht das richtige Werkzeug.

Fehlersuche

Es kommt nichts an

1. Ist der Haken bei *Webhook aktivieren* gesetzt und wurde **gespeichert**?
2. Was sagt **Testnachricht senden**? Die Statuszeile darunter nennt die Ursache im Klartext.
3. Schauen Sie unter **Übersicht** → **Gerätestatus** → **Details** in die Zeile *Webhook* — dort stehen Zähler für gesendet/fehlgeschlagen und die letzte Fehlermeldung.

Häufige Meldungen und was sie bedeuten

Meldung	Ursache	Abhilfe
ESP_ERR_HTTP_CONNECT	Server nicht erreichbar	IP und Port prüfen; steht der Server im selben Netz?
HTTP 404	URL stimmt nicht	Pfad in der Ziel-URL prüfen
HTTP 401 / HTTP 403	Ihr Server weist die Anfrage ab	Geheimnis eingetragen? Braucht Ihr Server eine Anmeldung?
ESP_ERR_ESP_TLS_...	Zertifikatsproblem bei https://	Selbstsigniertes Zertifikat wird nicht akzeptiert — http:// im lokalen Netz nutzen
ESP_ERR_TIMEOUT	Server antwortet zu langsam	Zeitlimit erhöhen oder die Verarbeitung hinter die Antwort legen

Meldungen kommen, aber die Zeit stimmt nicht

Prüfen Sie unter **Übersicht** → **Gerätestatus**, ob die Uhrzeit synchronisiert ist. Ohne NTP-Zugang bleibt das Feld `timestamp_ms` leer oder enthält eine falsche Zeit.

Für Entwickler: der API-Endpunkt

Die Testfunktion der Weboberfläche lässt sich auch direkt aufrufen:

```
POST /api/interfaces/webhook/test
Authorization: Bearer <token>
```

Antwort: `{"status":"ok","httpStatus":200,"error":""}`

Die Konfiguration liegt in der Gerätekonfiguration unter `config.interfaces.webhook`:

```
{
  "enabled": true,
  "url": "https://beispiel.de/hook",
  "secret": "mein-geheimnis",
  "timeout_ms": 5000,
  "events": { "outputs": true, "inputs": true, "boot": true }
}
```

Sie kann über `GET/POST /api/config` gelesen und geschrieben werden — siehe die REST-API-Dokumentation.

Zähler und letzter Fehler stehen in `GET /api/system/status` unter `interfaces.webhook`.